SIG Arbres — Documentation technique

BUREAU D'ÉTUDES · MOISSY-CRAMAYEL · M. JEAN-MARIE PRÉVOST

📅 Mars 2026 📄 Un seul fichier HTML 📡 Sans serveur requis

📱 Mobile & desktop

📋 Sommaire

01 Ce que fait l'application
02 La demande de départ
03 Les langages utilisés
04 Les fonctions développées
05 Personnaliser pour Moissy-Cramayel
06 Déploiement sur un serveur
07 Personnaliser les couleurs & l'apparence
08 Pistes d'évolution

Ce que fait l'application

Jean-Marie, l'application que j'ai construite pour vous est un **outil d'inventaire d'arbres sur le terrain**, utilisable directement depuis un smartphone, une tablette ou un ordinateur. Son nom technique est *SIG Arbres* — SIG signifie *Système d'Information Géographique*.

En termes simples : vous êtes sur le terrain, face à un arbre, vous ouvrez l'application dans le navigateur de votre téléphone, vous touchez la carte à l'endroit de l'arbre (ou vous activez le GPS), vous remplissez un formulaire rapide, et le point est enregistré immédiatement sur la carte et dans la mémoire de votre appareil. Plus besoin de carnet papier ni de logiciel lourd.

🟠 🟡 🟢 🌳 SIG Arbres — Inventaire de terrain

📍 Carte interactive (Leaflet)	📋 Formulaire espèce / état / hauteur
🛰️ Vue satellite Google disponible	📂 Liste de tous les arbres saisis
🗺️ Vue OpenStreetMap disponible	✏️ Modification / suppression
➕ Mode ajout point (clic ou GPS)	📤 Export CSV (Excel)
🌳 Marqueurs arbres personnalisés	📷 Export image A4 paysage

💡 **Aucune installation requise**

L'application tient dans un seul fichier `index.html`. Il suffit de l'ouvrir dans un navigateur web (Chrome, Safari, Firefox…) — sur téléphone, tablette ou ordinateur. Aucun logiciel à installer.

02

🎯 La demande de départ et comment j'y ai répondu

La demande initiale était de créer un outil de terrain léger, utilisable par le bureau d'études, pour géolocaliser et qualifier des arbres sans dépendre d'un logiciel SIG professionnel lourd (type QGIS ou ArcGIS). Voici comment j'ai traduit chaque besoin en fonctionnalité concrète :

BESOIN EXPRIMÉ	CE QUE J'AI DÉVELOPPÉ
Travailler sur le terrain avec un smartphone	Interface responsive, adaptée portrait et paysage, avec des boutons suffisamment grands pour les doigts
Placer un arbre sur une carte	Clic sur la carte en « mode ajout » ou GPS automatique, marqueur rouge en attente de validation
Saisir les caractéristiques de l'arbre	Formulaire structuré : type, espèce latine, date de plantation, hauteur, état sanitaire, observations libres
Base d'espèces d'arbres française	Menu déroulant avec +80 espèces classées par famille (Chêne, Tilleul, Pin, Cèdre…) et possibilité de saisir une espèce non listée
Récupérer les données	Export CSV (compatible Excel), partage par e-mail ou WhatsApp, export image carte A4 paysage (JPEG)
Retravailler les données entre deux sessions	Sauvegarde automatique dans le navigateur (localStorage) — les arbres sont toujours là à la réouverture
Utiliser en déplacement sans connexion	Application 100 % autonome — seules les tuiles de carte nécessitent du réseau ; les données restent locales

BESOIN EXPRIMÉ	CE QUE J'AI DÉVELOPPÉ
Identité visuelle bureau d'études	Palette forêt/écorce, typographie soignée, copyright « Bureau d'Études — M. Prévost » en pied de page

🏢 **Contexte : Moissy-Cramayel, Seine-et-Marne**

La carte est centrée par défaut sur les coordonnées `48.6167° N, 2.5833° E` — c'est exactement le centre de Moissy-Cramayel. Dès que vous ouvrez l'application, vous tombez sur votre territoire.

03 💻 Les langages de code utilisés

L'application repose sur trois langages qui constituent la base de toute page web. Je les ai tous mis dans un seul fichier, ce qui rend le déploiement très simple.

🏗️ HTML5 — Structure de la page

🎨 CSS3 — Apparence visuelle

⚙️ JavaScript — Fonctionnement interactif

HTML5 — la structure

C'est le squelette de la page. J'ai créé avec HTML tous les éléments visibles : l'en-tête avec le logo et les statistiques, la carte, le panneau latéral avec ses onglets, le formulaire de saisie, la liste des arbres, les modales (fenêtres flottantes) pour l'édition et l'export. En termes simples, HTML décrit *quoi* afficher, mais pas *comment* ça ressemble.

CSS3 — l'apparence

C'est la partie qui donne son allure à l'application : les couleurs forêt, les polices, les tailles, les animations, l'adaptation mobile. J'ai défini une

palette de couleurs nommées (comme `--forest`, `--leaf`, `--amber`...) que je réutilise partout — si vous changez une couleur, elle se met à jour dans toute l'application automatiquement. J'ai aussi écrit des règles spéciales pour que l'affichage s'adapte automatiquement à la taille de l'écran (ce qu'on appelle le *responsive design*).

JavaScript — le comportement

C'est le cerveau de l'application. Tout ce qui « fait quelque chose » est écrit en JavaScript : le clic sur la carte, la géolocalisation GPS, la sauvegarde, l'export CSV, la génération de l'image, etc. Je n'ai utilisé aucun framework externe compliqué — du JavaScript pur, lisible et modifiable directement.

Bibliothèques externes

J'ai intégré deux outils tiers pour éviter de réinventer la roue :

BIBLIOTHÈQUE	VERSION	RÔLE
Leaflet.js	1.9.4	Bibliothèque de carte interactive open source. Elle affiche les tuiles OpenStreetMap et Google Satellite, gère le zoom, les marqueurs, les popups. C'est la référence mondiale pour les cartes web légères.
html2canvas	1.4.1	Permet de prendre une « photo » de la carte telle qu'elle apparaît à l'écran et de la transformer en fichier JPEG téléchargeable. C'est ce qui alimente la fonction d'export image A4.

🌐 **Ces bibliothèques sont chargées depuis un CDN**

Elles ne sont pas stockées dans le fichier lui-même mais téléchargées automatiquement depuis internet à l'ouverture de l'application. Si vous travaillez hors ligne, il faudra les intégrer localement (voir section Déploiement).

04

⚙️ Les fonctions développées — expliquées simplement

Voici l'ensemble des fonctions JavaScript que j'ai écrites, regroupées par thème, avec une explication accessible pour quelqu'un qui ne code pas :

🗺️ Initialisation et gestion de la carte

`Initialisation Leaflet` carte

Crée la carte centrée sur Moissy-Cramayel au zoom 14. Active les contrôles de zoom. Deux fonds de carte disponibles : OpenStreetMap (plan) et Google Satellite.

`Sélecteur de fond de carte` carte

Les boutons OSM / Satellite en bas à gauche de la carte. Quand vous cliquez, l'ancien fond est retiré, le nouveau est chargé, et une notification apparaît brièvement en bas d'écran.

`makeIcon(color)` visuel

Crée l'icône d'arbre verte visible sur la carte. Je l'ai dessinée en SVG (dessin vectoriel) : un feuillage vert en ellipses superposées et un tronc marron. Elle s'adapte à n'importe quelle couleur si on la change.

`makeUserIcon()` visuel

Crée le point bleu/vert qui représente votre position GPS en temps réel sur la carte. Un cercle vert avec un halo pulsant pour être facilement repérable.

📍 Placement et géolocalisation

`Mode ajout (clic carte)` saisie

Quand vous activez le bouton « ➕ Ajouter point », le curseur de la carte passe en croix. Le prochain clic pose un marqueur rouge temporaire (en attente de validation) et affiche les coordonnées dans le formulaire.

`locateUser()` GPS

Utilise le GPS du téléphone (ou la géolocalisation du navigateur) pour trouver votre position précise. La carte se centre sur vous, un marqueur vert apparaît, et la précision (ex. ±5 m) est affichée. Fonctionne en temps réel —

si vous vous déplacez, le marqueur suit.

`makePendingMarker()` visuel

Pose le cercle rouge clignotant sur la carte quand un point a été cliqué mais pas encore enregistré dans le formulaire. C'est un indicateur visuel : « attention, ce point n'est pas encore sauvegardé ».

`setCoordDisplay()` formulaire

Met à jour les deux cases « Latitude » et « Longitude » en haut du formulaire avec les coordonnées du point cliqué ou GPS.

📋 Formulaire et saisie

`doSave()` cœur

Fonction centrale : elle lit toutes les valeurs du formulaire, crée un objet « arbre » avec un identifiant unique (horodatage), l'ajoute à la liste, le sauvegarde, place le marqueur sur la carte et met à jour les statistiques.

`Enregistrer & suivant` terrain

Variante de la sauvegarde prévue pour le terrain : après avoir enregistré un arbre, le formulaire se vide automatiquement, le mode ajout se réactive, et vous êtes prêt à placer le point suivant en un seul geste.

`clearForm()` formulaire

Vide tous les champs du formulaire : type, espèce, date, hauteur, état, notes, et remet les coordonnées à zéro. Supprime aussi le marqueur rouge temporaire.

`initDateSelects()` formulaire

Gère le sélecteur de date en trois parties (jour / mois / année) et le pré-remplit automatiquement avec la date du jour à l'ouverture.

📂 Gestion des espèces (nomenclature botanique)

`onVarieteSelect()` espèce

Gère le menu déroulant d'espèces. Si vous sélectionnez « ✏ Autre — saisir manuellement », un champ de texte

`getVarieteValue()` espèce

Récupère la valeur de l'espèce saisie — soit le nom latin depuis le menu, soit le texte libre tapé à la main.

libre apparaît en dessous pour que vous puissiez taper une espèce non listée.

`setVarieteValue()` espèce

Utilisée lors de l'édition d'un arbre existant : remet la bonne espèce dans le formulaire. Si l'espèce ne figure pas dans la liste standard, elle active automatiquement le champ de saisie libre.

📃 Liste et édition

`renderList()` liste

Génère la liste de toutes les fiches d'arbres dans le panneau latéral. Chaque carte affiche l'emoji de l'espèce, le nom, les coordonnées, et les boutons ✏️ (modifier) et × (supprimer).

`selectTree(id)` liste

Quand vous cliquez sur un arbre dans la liste, la carte se centre sur lui, son popup s'ouvre, et la carte surligne la fiche correspondante.

`openEditModal()` édition

Ouvre la fenêtre flottante d'édition et la pré-remplit avec toutes les données de l'arbre sélectionné : espèce, date, hauteur, état, notes.

`saveEditModal()` édition

Sauvegarde les modifications faites dans la fenêtre d'édition, met à jour la carte et la liste, et ferme la fenêtre.

`deleteTree()` données

Supprime définitivement un arbre après confirmation. Retire le marqueur de la carte, la fiche de la liste, et met à jour la sauvegarde locale.

💾 Sauvegarde et export

`saveLocal()` stockage

Sauvegarde tous les arbres dans la mémoire permanente du navigateur (localStorage). Les données restent disponibles même après avoir fermé la page, éteint le téléphone, ou redémarré. Elles sont liées au navigateur et à l'appareil utilisé.

`buildCSV()` export

Génère un fichier texte structuré (CSV) avec toutes les données des arbres : id, type, espèce, date de plantation, latitude, longitude, hauteur, état sanitaire, observations, date de saisie. Ce fichier s'ouvre directement dans Excel ou LibreOffice Calc.

`buildTextSummary()` export

Génère un résumé texte lisible de l'inventaire, utilisé pour l'envoi par e-mail : nombre d'arbres, date, liste avec coordonnées.

Partage natif (Web Share API) mobile

Sur mobile, active le menu de partage natif du téléphone pour envoyer le fichier CSV directement par WhatsApp, AirDrop, e-mail, Google Drive, etc.

📷 Export image A4

`openPrintOverlay()` export

Ouvre l'écran d'aperçu de l'export image. Une mini-carte Leaflet se crée automatiquement dans une composition A4 paysage, avec en-tête, légende, statistiques et pied de page.

`generateJpeg(mode)` export

Utilise html2canvas pour prendre une capture de la composition A4, puis propose soit de télécharger le JPEG, soit de le partager via le menu natif du téléphone. Le fichier est nommé automatiquement avec la date du jour.

📱 Interface mobile

`togglePanel()` mobile

Masque ou affiche le panneau latéral (formulaire + liste) pour donner plus d'espace à la carte. Sur mobile portrait, le panneau se glisse vers le bas comme un tiroir.

Drag handle (tiroir mobile) mobile

Sur mobile portrait, vous pouvez glisser le bord supérieur du panneau avec le doigt pour l'agrandir ou le réduire à la hauteur que vous

souhaitez, exactement comme un tiroir physique.

`showToast(msg)` `interface`

Affiche un message d'information discret en bas de l'écran pendant 2,5 secondes (ex. « 🌳 Chêne enregistré ! »). C'est la façon la moins intrusive de confirmer une action sans interrompre le travail.

`updateStats()` `interface`

Met à jour en temps réel les deux compteurs en haut de l'écran : nombre total d'arbres saisis, et nombre d'espèces différentes.

📥 Import CSV

`Importer un CSV` `import`

Permet de charger un fichier CSV précédemment exporté pour reprendre un inventaire sur un autre appareil. L'application parse le fichier, recrée tous les arbres en mémoire et repose tous les marqueurs sur la carte.

05 🏙️ Personnaliser pour Moissy-Cramayel et le bureau d'études

Voici les lignes précises à modifier dans le fichier `index.html` pour adapter l'application à votre contexte. Vous pouvez ouvrir le fichier avec un éditeur de texte simple (Notepad++, Visual Studio Code, ou même le Bloc-notes de Windows).

1. Nom et identité de l'application

🏷️ Titre dans l'onglet du navigateur

Cherchez la ligne :

```
<title>SIG Arbres — Inventaire de terrain</title>
```

Remplacez par :

```
<title>SIG Arbres — Moissy-Cramayel — BET</title>
```

🌳 Sous-titre affiché dans l'en-tête

Cherchez la ligne (vers le bas du fichier, dans la section HTML) :

```
Inventaire de terrain — Bureau d'Études
```

Remplacez par :

```
Ville de Moissy-Cramayel — Service Espaces Verts
```

2. Centre de la carte — Moissy-Cramayel

📍 Centrage au démarrage

La carte est déjà centrée sur Moissy-Cramayel. Pour ajuster encore plus précisément (par exemple sur un parc ou un quartier particulier), cherchez :

```
center: [48.6167, 2.5833],
zoom: 14
```

Ajustez le zoom : **14** = vue de la commune, **16** = vue de quartier, **18** = vue parcelle. Pour connaître les coordonnées exactes d'un lieu, ouvrez Google Maps, faites un clic droit, et les coordonnées apparaissent.

3. Copyright et pied de page

© Mention dans l'export image

Cherchez (deux occurrences dans le fichier) :

```
© 2026 Bureau d'Études — M. Prévost
```

Remplacez par votre mention officielle, par exemple :

```
© 2026 Ville de Moissy-Cramayel — Service Espaces Verts
```

4. Noms dans les exports CSV et e-mail

📤 Nom des fichiers exportés

Cherchez :

```
inventaire_BET_Prevost_
```

Remplacez par :

```
inventaire_MoissyCramayel_
```

Le fichier téléchargé s'appellera alors `inventaire_MoissyCramayel_2026-03-20.csv`.

5. Résumé textuel de l'inventaire (e-mail)

📧 **Texte envoyé par e-mail**

Cherchez la ligne :

```
'🌳 Inventaire BET M. Prévost\n'
```

Remplacez par :

```
'🌳 Inventaire arbres — Moissy-Cramayel\n'
```

✅ **Résumé des 5 modifications pour Moissy-Cramayel**

1. Titre de l'onglet — 2. Sous-titre en-tête — 3. Centrage carte — 4. Copyright pied de page — 5. Nom des fichiers exportés. Ces 5 changements suffisent à personnaliser complètement l'outil pour votre collectivité.

06 🚀 Déployer sur un serveur web

L'application ne nécessite qu'un **serveur web statique** — c'est-à-dire un serveur capable de distribuer des fichiers HTML. Pas besoin de base de données, pas besoin de PHP, pas besoin de Node.js. C'est l'hébergement le plus simple et le moins cher qui existe.

Prérequis serveur minimaux

ÉLÉMENT	REQUIS	DÉTAIL
Type d'hébergement	✅ Oui	Tout hébergement web standard (OVH, PlanetHoster, Infomaniak, o2switch…)
PHP / MySQL	❌ Non	Aucune base de données nécessaire — les données sont stockées dans le navigateur
HTTPS	⚠️ Recommandé	Le GPS du téléphone est bloqué sur HTTP non sécurisé par les navigateurs modernes. Un certificat SSL (gratuit avec Let's Encrypt) est nécessaire pour la géolocalisation.
Espace disque	✅ Minimal	Le fichier `index.html` fait environ 250 ko — n'importe quel hébergement convient
Bande passante	✅ Faible	Seules les tuiles de carte (images OSM/Google) consomment du réseau, côté navigateur
Nom de domaine	✅ Optionnel	L'application peut vivre sur un sous-domaine ou dans un dossier existant

Étapes de déploiement

1. **Préparez votre fichier**

   Faites les personnalisations décrites en section 5 (nom, coordonnées, copyright). Enregistrez le fichier sous le nom `index.html`.

2. **Connectez-vous à votre hébergement**

   Via le gestionnaire de fichiers de votre panneau d'hébergement (cPanel, Plesk, DirectAdmin…) ou via un logiciel FTP comme FileZilla.

3. **Déposez le fichier**

   Copiez `index.html` dans le dossier `public_html` (ou `www`, ou un sous-dossier de votre choix). Un seul fichier suffit.

4. **Activez HTTPS**

   Dans le panneau de votre hébergeur, activez le certificat SSL gratuit Let's Encrypt (proposé par tous les hébergeurs modernes en un clic). Cela active l'adresse `https://` indispensable pour le GPS.

5. **Testez sur mobile**

   Ouvrez l'adresse sur votre téléphone, autorisez la géolocalisation, et testez le cycle complet : GPS → formulaire → enregistrement → export CSV.

> ⚠️ **Important : les données restent sur l'appareil**
>
> Les arbres saisis sont stockés dans le navigateur de chaque appareil (`localStorage`). Si deux agents de terrain utilisent l'application sur deux téléphones différents, leurs inventaires ne se synchronisent pas automatiquement. Pour partager les données, chacun exporte son CSV et vous les fusionnez dans Excel. Pour une synchronisation en temps réel, il faudrait une évolution vers une base de données (voir section 8).

Option : utilisation hors ligne complète (PWA)

Il est possible d'aller plus loin et de transformer l'application en *Progressive Web App* (PWA), ce qui permet de l'installer sur l'écran d'accueil du téléphone comme une vraie application, et de la faire fonctionner entièrement hors connexion (y compris les tuiles de carte en cache). C'est une évolution possible sur demande.

07

🎨 Personnaliser les couleurs et l'apparence

J'ai organisé toutes les couleurs en variables au début du CSS. Il suffit de modifier ces quelques lignes pour changer entièrement l'aspect de l'application :

```
/* Cherchez ce bloc au début du fichier, dans la balise <style> */
:root {
  --forest:   #1a2e1a;  /* Fond principal (vert très sombre)     */
  --moss:     #2e4a2e;  /* Fond secondaire (vert mousse)         */
  --fern:     #3d6b3d;  /* Bordures et éléments discrets        */
  --leaf:     #5a9a4a;  /* Couleur principale (vert feuille)    */
  --sage:     #a8c897;  /* Textes sur fond sombre               */
  --cream:    #f5f0e8;  /* Texte principal clair                */
  --amber:    #d4891a;  /* Bouton export (orange ambre)         */
  --rust:     #c05a2a;  /* Bouton danger / suppression          */
}
```

🎨 Exemple : thème bleu municipal

Pour donner un aspect institutionnel bleu-gris (couleurs de collectivité), remplacez par exemple :

```
  --forest:   #1a2540;  /* bleu nuit  */
  --moss:     #2a3a5e;  /* bleu marine */
  --leaf:     #4a7fc1;  /* bleu vif   */
  --sage:     #a0bcdd;  /* bleu ciel  */
```

🔤 Changer les polices de caractères

L'application utilise deux polices Google Fonts :

`DM Serif Display` → les titres élégants en serif

`DM Mono` → les textes techniques en chasse fixe

Pour les remplacer, modifiez la ligne `<link href="https://fonts.googleapis.com/...` en début de fichier, et remplacez les occurrences de `'DM Mono', monospace` et `'DM Serif Display', serif` dans le CSS.

🌳 Changer l'icône d'arbre sur la carte

L'icône est dessinée en SVG dans la fonction `makeIcon()`. Vous pouvez la remplacer par une image PNG/SVG de votre choix en changeant :

```
// Remplacer makeIcon() par une image externe :
return L.icon({
  iconUrl:    './mon-icone-arbre.png',
  iconSize:   [32, 40],
  iconAnchor: [16, 38]
});
```

📍 Changer le fond de carte par défaut

Par défaut la carte s'ouvre sur OpenStreetMap. Pour démarrer sur Google Satellite, cherchez la ligne :

```
layerOSM.addTo(map);
let activeLayer = layerOSM;
```

Et remplacez par :

```
layerIGN.addTo(map);
let activeLayer = layerIGN;
```

📋 Ajouter ou retirer des champs dans le formulaire

Pour ajouter un champ (ex. « Numéro de patrimoine »), ajoutez dans le HTML du formulaire :

```
<label for="field-patrimoine">N° Patrimoine</label>
<input type="text" id="field-patrimoine" placeholder="ex. ARB-0042" />
```

Puis dans la fonction `doSave()`, récupérez la valeur en ajoutant :

```
const patrimoine = document.getElementById('field-patrimoine').value;
```

Et incluez-la dans l'objet `tree` et dans la fonction `buildCSV()`.

08 🔭 Pistes d'évolution suggérées

L'application dans sa forme actuelle couvre l'essentiel d'un inventaire de terrain. Voici les évolutions les plus utiles que je pourrais développer si vous le souhaitez :

ÉVOLUTION	INTÉRÊT	COMPLEXITÉ
📸 **Photo de l'arbre**	Prendre une photo directement depuis l'application, stockée avec la fiche	Moyenne
☁️ **Synchronisation cloud**	Partager les données entre plusieurs agents de terrain en temps réel (via Firebase ou Supabase gratuits)	Élevée
📏 **Diamètre / circonférence**	Ajouter un champ de mesure de tronc, courant dans les inventaires patrimoniaux	Faible
		Faible

ÉVOLUTION	INTÉRÊT	COMPLEXITÉ
🔴 **Couleur par état sanitaire**	Marqueurs verts / orange / rouges selon l'état sanitaire (Bon / Moyen / Mauvais)	
📊 **Statistiques détaillées**	Tableau de bord avec répartition par espèce, par état, histogrammes	Moyenne
🗂️ **Zones / parcelles**	Délimiter des zones sur la carte et affecter chaque arbre à une zone	Élevée
📱 **Installation PWA**	Installer l'application sur l'écran d'accueil du téléphone, fonctionnement hors ligne complet avec cache des tuiles	Moyenne
🖨️ **Export PDF imprimable**	Générer un rapport PDF mis en page avec liste complète, carte et tableau récapitulatif	Élevée
🔢 **Numérotation automatique**	Attribuer un numéro de patrimoine incrémental à chaque arbre saisi	Faible

💬 **Pour toute évolution ou adaptation**

Cette documentation, comme l'application elle-même, peut évoluer. N'hésitez pas à me soumettre vos besoins terrain : les ajustements listés ci-dessus peuvent être intégrés progressivement dans le même fichier unique, sans changer votre façon de déployer l'outil.

🔑 **Rappel des points clés**

Un seul fichier HTML

— pas d'installation, pas de dépendance.

HTTPS obligatoire

pour le GPS en production.

Les données restent dans le navigateur

— pensez à exporter régulièrement en CSV.

5 lignes à modifier

pour personnaliser l'outil à Moissy-Cramayel.

Tout hébergement web standard

convient au déploiement.